Web Programming

Rodrigo Martins Pagliares

Last update: 11/01/2025

Web Programming

Rodrigo Martins Pagliares

JavaScript
Dynamic client-side scripting

JavaScript first steps

Storing the information you need

(Variables)

Lecture: Storing the information you need - Variables
Module: JavaScript first steps

Objective of the lecture

Objective: To gain **familiarity** with the **basics** of **JavaScript variables**.

Introduction

Introduction

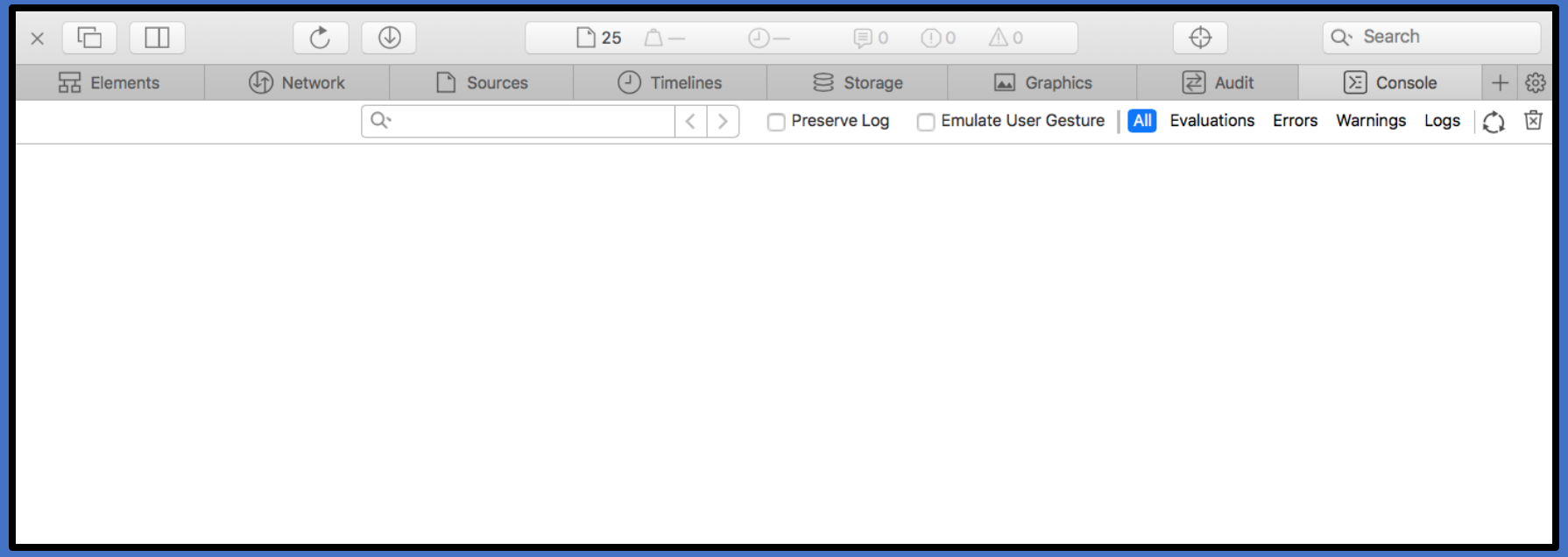
- In this **lecture**, we will **get down** to the real **basics**, looking at **how to work** with the **most basic building blocks** of **JavaScript**.
  - ✓ **Variables**.

Tools you need

Tools you need

- **Throughout** this **lecture**, you'll be **asked** to **type in lines of code** to **test** your **understanding** of the **content**.

- If you are **using** a **desktop browser**, the **best place** to **type** your **sample code** is your **browser's JavaScript console**.



What is a variable?

What is a variable?

```
1 <button>Press me</button>
```

HTML

```
1 const button = document.querySelector('button');
2
3 button.onclick = ( ) => {
4   let name = prompt('What is your name?');
5   alert('Hello ' + name + ', nice to see you!');
6 }
```

JavaScript

▸ CSS building blocks
▸ Styling text
▸ CSS layout

JavaScript — Dynamic client-side scripting

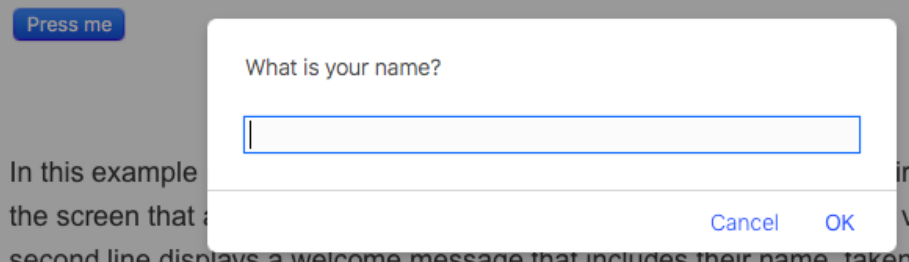
▾ JavaScript first steps
- JavaScript first steps overview
- What is JavaScript?
- A first splash into JavaScript
- What went wrong? Troubleshooting JavaScript
- Storing the information you nee… — Variables
- Basic math in JavaScript — Numbers and operators
- Handling text — Strings in JavaScript
- Useful string methods
- Arrays
- Assessment: Silly story generator

▸ JavaScript building blocks
▸ Introducing JavaScript objects
▸ Asynchronous JavaScript

```
<button>Press me</button>
```

```
const button = document.querySelector('button');

button.onclick = function() {
  let name = prompt('What is your name?');
  alert('Hello ' + name + ', nice to see you!');
}
```



In this example … irst line pops a box up on the screen that … value in a variable. The second line displays a welcome message that includes their name, taken from the variable value.

To understand why this is so useful, let's think about how we'd write this example without using a variable. It would end up looking something like this:

```
let name = prompt('What is your name?');

if (name === 'Adam') {
  alert('Hello Adam, nice to see you!');
} else if (name === 'Alan') {
  alert('Hello Alan, nice to see you!');
} else if (name === 'Bella') {
  alert('Hello Bella, nice to see you!');
```

https://mzl.la/3bH8xAB

▸ CSS building blocks
▸ Styling text
▸ CSS layout

JavaScript — Dynamic client-side scripting

▾ JavaScript first steps
- JavaScript first steps overview
- What is JavaScript?
- A first splash into JavaScript
- What went wrong? Troubleshooting JavaScript
- Storing the information you need — Variables
- Basic math in JavaScript — Numbers and operators
- Handling text — Strings in JavaScript
- Useful string methods
- Arrays
- Assessment: Silly story generator

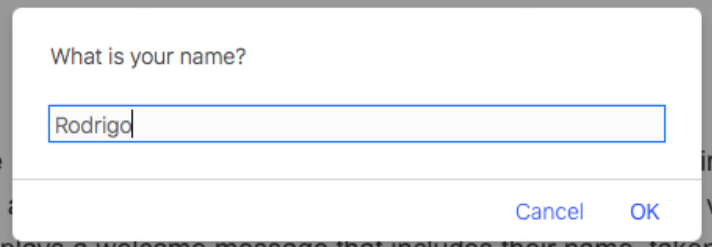
▸ JavaScript building blocks
▸ Introducing JavaScript objects
▸ Asynchronous JavaScript

```
<button>Press me</button>
```

```
const button = document.querySelector('button');

button.onclick = function() {
  let name = prompt('What is your name?');
  alert('Hello ' + name + ', nice to see you!');
}
```

Press me



In this example [illegible] first line pops a box up on the screen that a[illegible] value in a variable. The second line displays a welcome message that includes their name, taken from the variable value.

To understand why this is so useful, let's think about how we'd write this example without using a variable. It would end up looking something like this:

```
let name = prompt('What is your name?');

if (name === 'Adam') {
  alert('Hello Adam, nice to see you!');
} else if (name === 'Alan') {
  alert('Hello Alan, nice to see you!');
} else if (name === 'Bella') {
  alert('Hello Bella, nice to see you!');
```

▸ CSS building blocks

▸ Styling text

▸ CSS layout

JavaScript — Dynamic client-side scripting

▾ JavaScript first steps

- JavaScript first steps overview
- What is JavaScript?
- A first splash into JavaScript
- What went wrong? Troubleshooting JavaScript
- Storing the information you need — Variables
- Basic math in JavaScript — Numbers and operators
- Handling text — Strings in JavaScript
- Useful string methods
- Arrays
- Assessment: Silly story generator

▸ JavaScript building blocks

▸ Introducing JavaScript objects

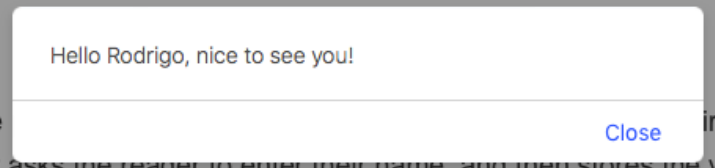
▸ Asynchronous JavaScript

```
<button>Press me</button>
```

```
const button = document.querySelector('button');

button.onclick = function() {
  let name = prompt('What is your name?');
  alert('Hello ' + name + ', nice to see you!');
}
```

Press me



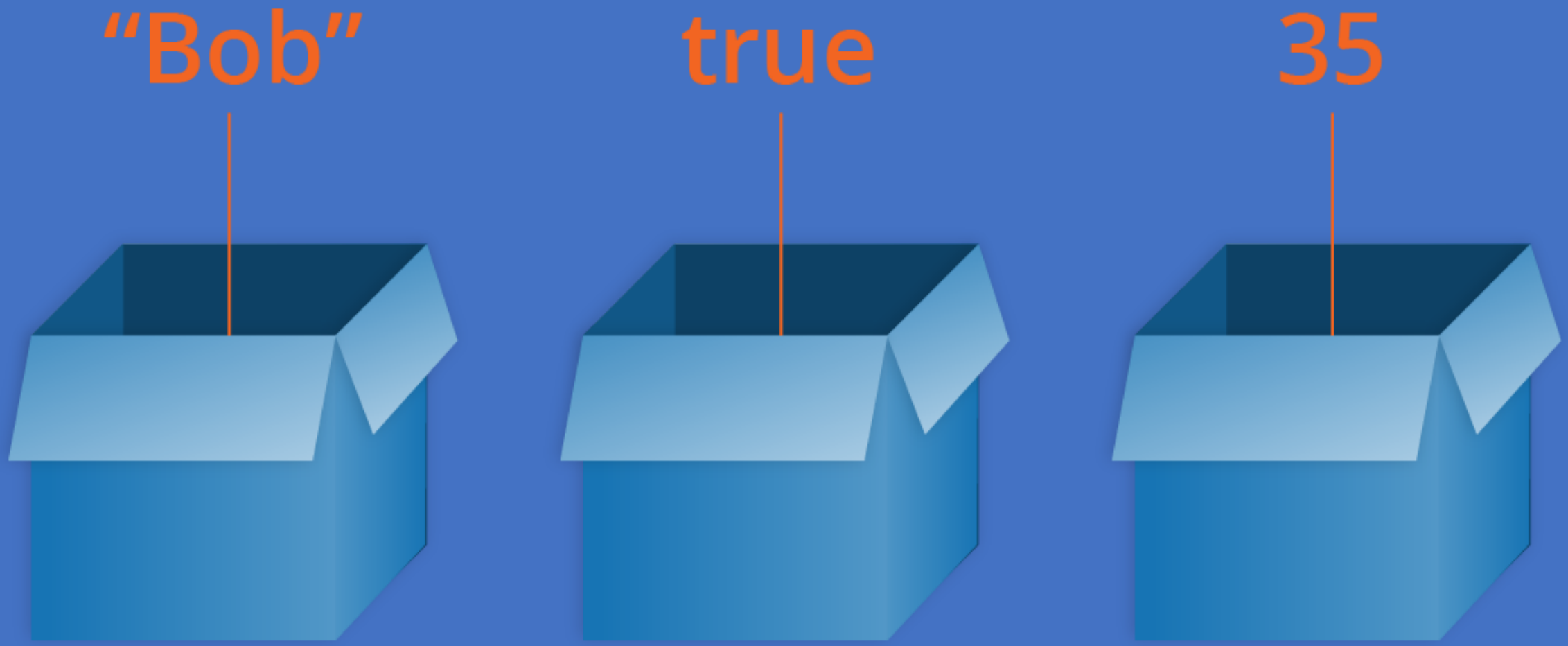
In this example … first line pops a box up on the screen that asks the reader to enter their name, and then stores the value in a variable. The second line displays a welcome message that includes their name, taken from the variable value.

To understand why this is so useful, let's think about how we'd write this example without using a variable. It would end up looking something like this:

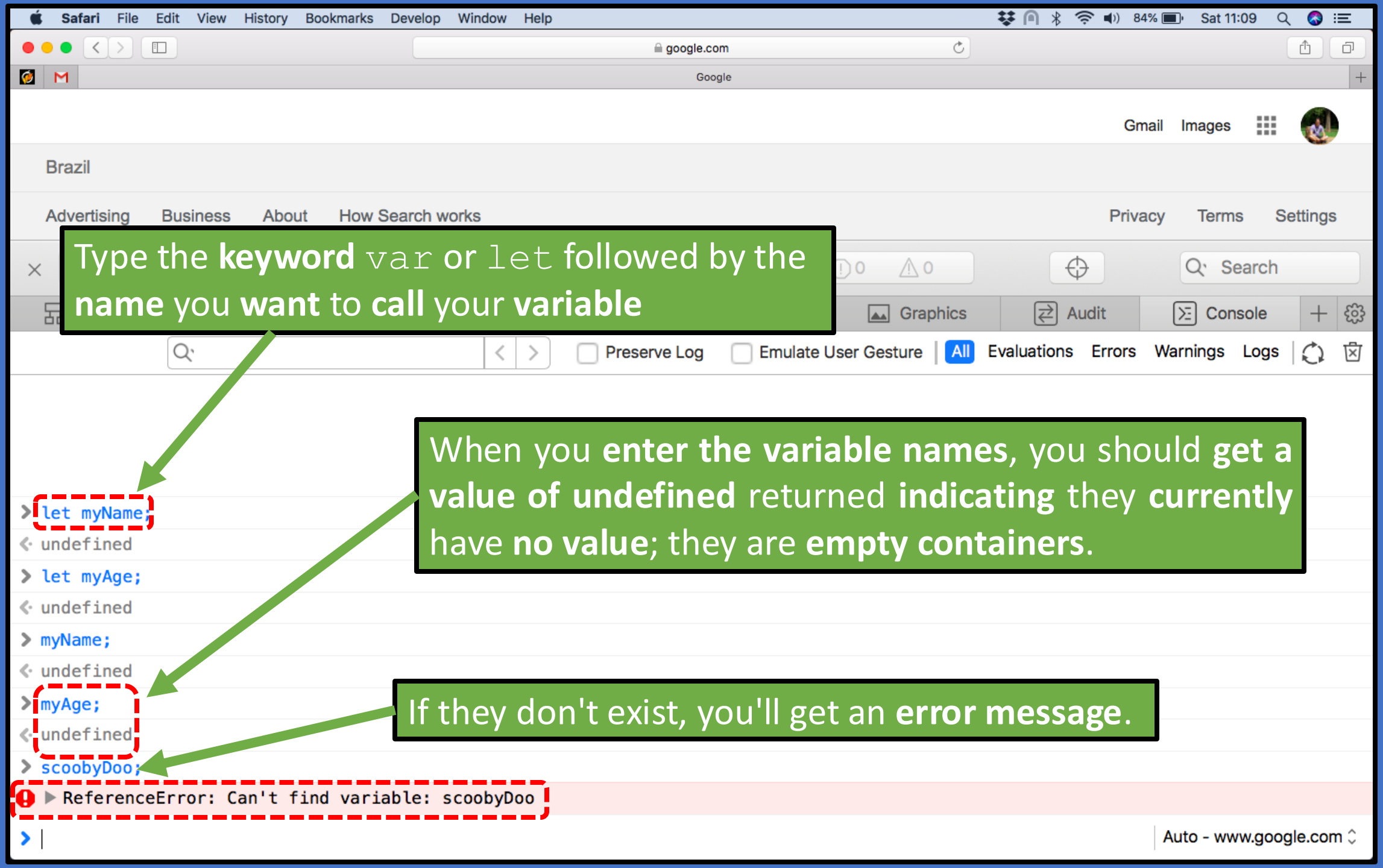
```
let name = prompt('What is your name?');

if (name === 'Adam') {
  alert('Hello Adam, nice to see you!');
} else if (name === 'Alan') {
  alert('Hello Alan, nice to see you!');
} else if (name === 'Bella') {
  alert('Hello Bella, nice to see you!');
```

Variables



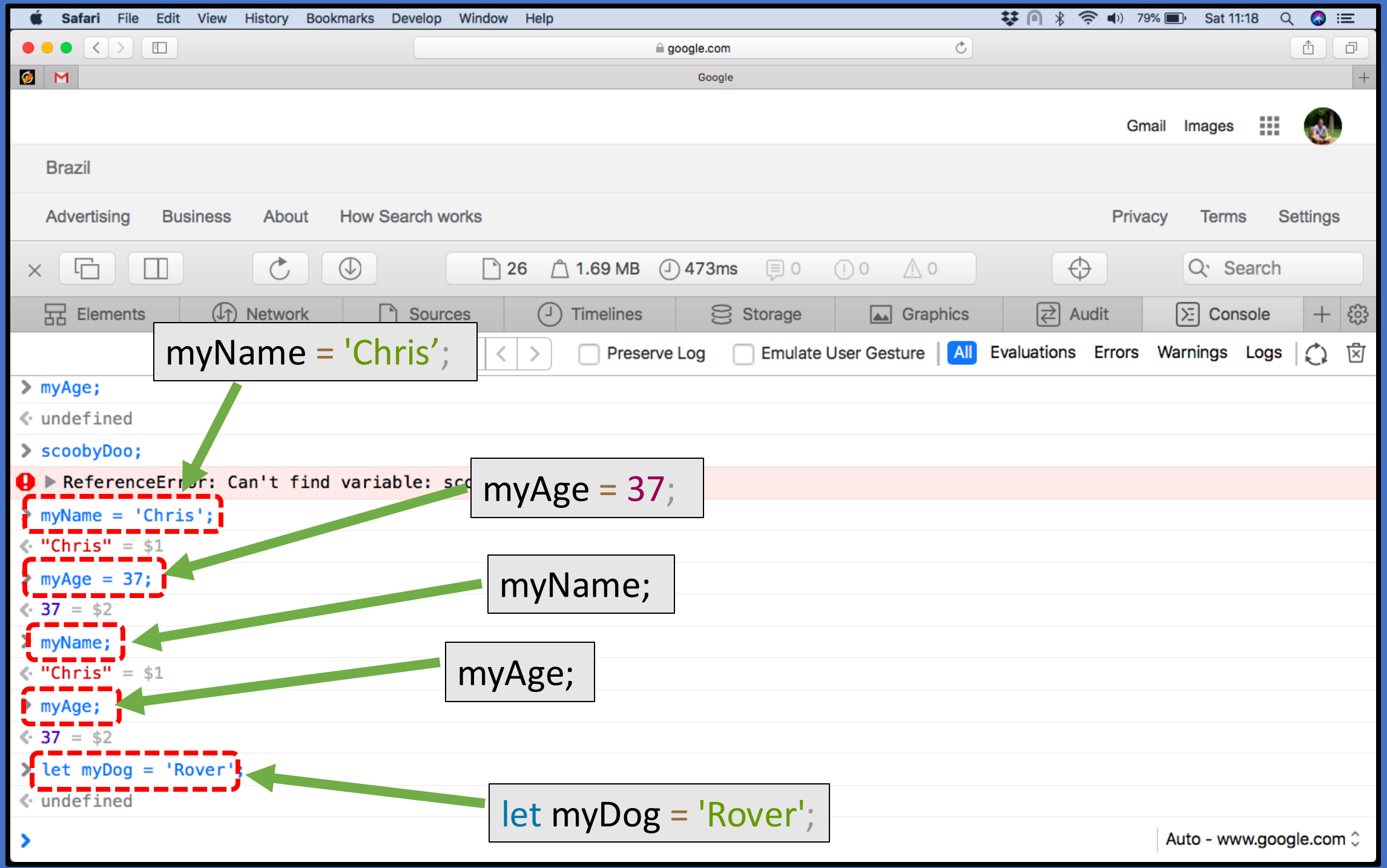
Declaring a variable


Safari File Edit View History Bookmarks Develop Window Help
84% Sat 11:09
google.com
Google
Gmail Images
Brazil
Advertising Business About How Search works
Privacy Terms Settings
Search
Graphics Audit Console
Preserve Log Emulate User Gesture All Evaluations Errors Warnings Logs
Type the keyword var or let followed by the name you want to call your variable
When you enter the variable names, you should get a value of undefined returned indicating they currently have no value; they are empty containers.
If they don't exist, you'll get an error message.
let myName;
undefined
let myAge;
undefined
myName;
undefined
myAge;
undefined
scoobyDoo;
ReferenceError: Can't find variable: scoobyDoo
Auto - www.google.com

Note

Note: In **JavaScript**, all code **instructions** should **end** with a **semi-colon** (;) — your **code may work** correctly for **single lines**, but **probably won't** when you are **writing multiple lines** of **code** together. Try to get into the **habit** of **including** it.

Initializing a variable


Safari File Edit View History Bookmarks Develop Window Help
79% Sat 11:18
google.com
Google
Gmail Images
Brazil
Advertising Business About How Search works
Privacy Terms Settings
26 1.69 MB 473ms 0 0 0
Search
Elements Network Sources Timelines Storage Graphics Audit Console
Preserve Log Emulate User Gesture All Evaluations Errors Warnings Logs
myName = 'Chris';
myAge = 37;
myName;
myAge;
let myDog = 'Rover';
> myAge;
undefined
> scoobyDoo;
ReferenceError: Can't find variable: sc
> myName = 'Chris';
"Chris" = $1
> myAge = 37;
37 = $2
> myName;
"Chris" = $1
> myAge;
37 = $2
> let myDog = 'Rover';
undefined
Auto - www.google.com

The difference between `var` and `let`

The difference between `var` and `let`

- Back **when JavaScript was first created**, there was **only** `var`.
  - ✓Works **fine** in **most cases**, but it has **some issues** in the way it **works**.
    - Its **design** can **sometimes** be **confusing** or **downright annoying**.
- `let` was **created** in **modern versions** of **JavaScript**.
  - ✓A new **keyword** for **creating variables** that **works** somewhat **differently** to `var`, fixing its **issues** in the **process**.

The difference between `var` and `let`

- **First**, if you **write** a **multiline JavaScript program** that **declares** and **initializes** a **variable**, you can **actually declare** a **variable** with `var` **after** you **initialize it** and it will still **work**.
  - ✓It works **because** a **mechanism** called **hoisting**.
  - ✓**Hoisting** no **longer works** with `let`.

```
1  myName = 'Chris';
2
3  function logName() {
4    console.log(myName);
5  }
6
7  logName();
8
9  var myName;
```

Note

Hoisting won't work when typing **individual lines** into a **JavaScript console**, **just** when **running** multiple **lines** of **JavaScript** in a **web document**.

var hoisting

Because variable declarations (and declarations in general) are processed before any code is executed, declaring a variable anywhere in the code is equivalent to declaring it at the top. This also means that a variable can appear to be used before it's declared. This behavior is called "hoisting", as it appears that the variable declaration is moved to the top of the function or global code.

```
1  bla = 2;
2  var bla;
3
4  // ...is implicitly understood as:
5
6  var bla;
7  bla = 2;
```

For that reason, it is recommended to always declare variables at the top of their scope (the top of global code and the top of function code) so it's clear which variables are function scoped (local) and which are resolved on the scope chain.

It's important to point out that the hoisting will affect the variable declaration, but not its value's initialization. The value will be indeed assigned when the assignment statement is reached:

```
1  function do_something() {
2    console.log(bar); // undefined
3    var bar = 111;
```

https://mzl.la/361bsTS

The difference between `var` and `let`

- **Second**, when you use `var`, you can **declare** the **same variable** as **many times** as you like, but **with let you can't**.



```
1 let myName = 'Chris’;
2 let myName = 'Bob';
```

```
1 var myName = 'Chris’;
2 var myName = 'Bob';
```



```
1 let myName = 'Chris';
2 myName = 'Bob';
```



The difference between `var` and `let`

- There is **no reason to redeclare variables**.
  - ✓It just **makes things** more **confusing**.

- We **recommend** that you use `let` **as much as possible** in your **code**, **rather** than `var`.

Variable naming rules

Variable naming rules

- You should **stick** to just using **Latin characters** (0-9, a-z, A-Z) and the **underscore character**.
  - ✓**Don't** use **numbers** at the **start** of **variables**.
  - ✓This **causes** an **error**.
- A **safe convention** to **stick** to is so-called “**lower camel case**”.
  - ✓**Used** in this **course**.

Variable naming rules (cont.)

- **Variables** are **case sensitive**.
- **Avoid** using **JavaScript reserved words** as your **variable names**.
  - ✓`var`, `function`, `let`, **and** `for`**.**
  - ✓**Browsers recognize** them as **different code items**, and so you'll get **errors**.

Note

Note: You can find a **fairly complete list** of **reserved keywords** to avoid at **Lexical grammar — keywords**.

Keywords

Reserved keywords as of ECMAScript 2015

- `break`
- `case`
- `catch`
- `class`
- `const`
- `continue`
- `debugger`
- `default`
- `delete`
- `do`
- `else`
- `export`
- `extends`
- `finally`
- `for`
- `function`
- `if`
- `import`
- `in`
- `instanceof`
- `new`
- `return`
- `super`
- `switch`
- `this`
- `throw`
- `try`
- `typeof`
- `var`
- `void`
- `while`
- `with`
- `yield`

Future reserved keywords

The following are reserved as future keywords by the ECMAScript specification. They have no special functionality at present, but they might at some future time, so t
identifiers.

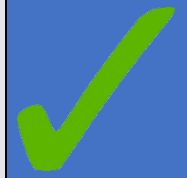
These are always reserved:

https://mzl.la/3cGNeR0

Variable naming rules

Good name examples

```
1  age
2  myAge
3  init
4  initialColor
5  finalOutputValue
6  audio1
7  audio2
```



Bad name examples

```
1  1
2  a
3  _12
4  myage
5  MYAGE
6  var
7  Document
8  skjfndskjfnbdskjfb
9  thisisareallylongstupidvariablenameman
```



Variable

Variable types

- Numbers
- Strings
- Booleans
- Arrays
- Objects

Numbers

```
1 let myAge = 17;
```

Arrays

```
1 let myNameArray = ['Chris', 'Bob', 'Jim'];
2 let myNumberArray = [10, 15, 40];
```

```
1 myNameArray[0]; // should return 'Chris'
2 myNumberArray[2]; // should return 40
```

Strings

```
1 let dolphinGoodbye = 'So long and thanks for all the fish';
```

Objects

```
1 let dog = { name : 'Spot', breed : 'Dalmatian' };
```

```
1 dog.name
```

Booleans

```
1 let iAmAlive = true;
```

```
1 let test = 6 < 3;
```

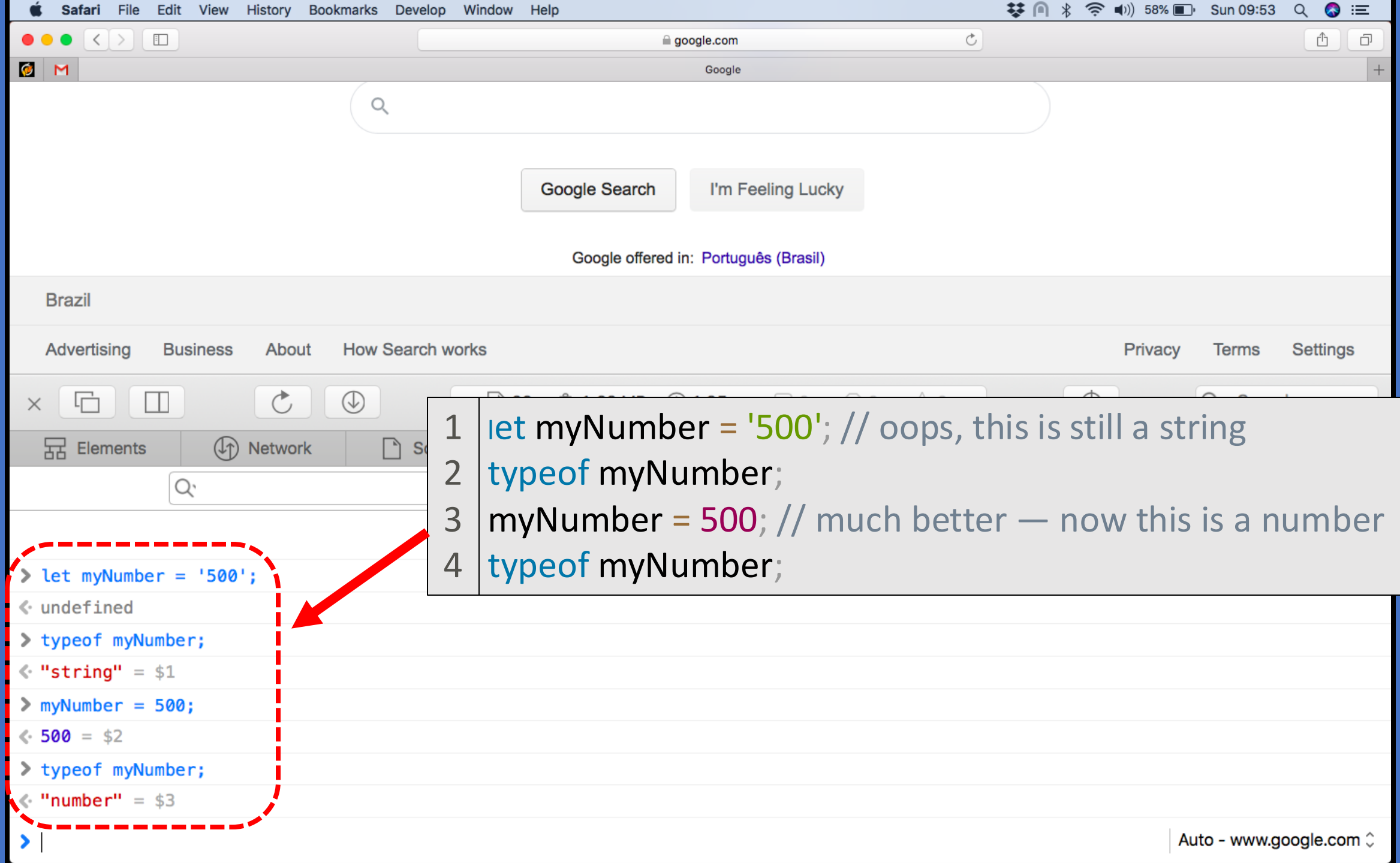
Dynamic typing

Dynamic typing

- **JavaScript** is a "**dynamically typed language**".
- This **means** that, **unlike** some **other languages**, you **don't need** to **specify** what **data type** a **variable** will **contain**.
  - ✓**Numbers**, **strings**, **arrays**, etc.

```
1 let myString = 'Hello';
```

The **browser treats** the **variable** as a **string**


Safari File Edit View History Bookmarks Develop Window Help
58% Sun 09:53
google.com
Google
Google Search
I'm Feeling Lucky
Google offered in: Português (Brasil)
Brazil
Advertising Business About How Search works
Privacy Terms Settings
Elements
Network
1 let myNumber = '500'; // oops, this is still a string
2 typeof myNumber;
3 myNumber = 500; // much better — now this is a number
4 typeof myNumber;
> let myNumber = '500';
undefined
> typeof myNumber;
"string" = $1
> myNumber = 500;
500 = $2
> typeof myNumber;
"number" = $3
Auto - www.google.com

Constants in JavaScript

Constants in JavaScript

- **Constant**
  - ✓A **value** that **once declared can't be changed**.
- There are **many reasons why** you'd **want** to **use constants**:
  - ✓**Security**
    - If a **third-party script changed** such **values** it **could cause problems**.
  - ✓**Debugging** and **code comprehension**
    - It is **harder** to **accidentally change values** that **shouldn't be changed** and **mess things up**.

Constants in JavaScript

- In the **early days** of **JavaScript**, **constants didn't exist**.
  - ✓**Introduced** in **ECMAScript 2015**.

- In **modern JavaScript**, we have the **keyword** `const`.



```
1  const daysInWeek = 7;
2  const hoursInDay = 24;
```

```
1  const daysInWeek = 7;
2  daysInWeek = 8;
```



Questions?